

Workflows at the Core

Executive Summary

This document details the architecture, design philosophy, and technical implementation of the Workflow Engine that powers Tuva ITSM's agentic automation capabilities. Unlike traditional workflow systems where humans orchestrate processes, our system inverts this relationship: an intelligent AI agent discovers, initializes, and executes workflows to fulfill user requests across diverse business domains.

The workflow engine represents our solution to the fundamental scalability paradox: how to build an agent capable of executing an unlimited number of operations while maintaining the flexibility to adapt each operation's process to client-specific requirements. This document chronicles the design decisions that transformed workflow execution from a conceptual abstraction into a production-ready system managing over seventy distinct business processes across multiple enterprise platforms.

The Core Problem: Scalable Agentic Automation

The Challenge

Building an AI assistant that can execute "hell lot of operations" presents three fundamental constraints:

1. **Unknown Scale:** We cannot predict how many operations the system will need to support in the future. The architecture must accommodate growth without fundamental redesign.
2. **Process Variability:** Even when the objective is identical (e.g., "create a ticket"), the process to achieve it may differ significantly between clients. One organization might require managerial approval, another might need additional validation steps, while a third might integrate with different backend systems.
3. **Product vs. Service Dichotomy:** As a product-based company, we need a generic system that remains consistent across all clients while simultaneously adapting to specialized requirements. We cannot build separate agents for each client.

The Philosophical Shift: Workflows as Executable Graphs

Our solution emerged from a critical insight: we needed to separate the "what" from the "how." The AI agent should understand what needs to be done (the objective), while workflows define how to do it (the process). This separation enables:

- **Scalability:** New operations are added as new workflows, not as new code in the agent
- **Customizability:** Each client can have tailored workflows without forking the codebase

- **Maintainability:** Business logic lives in configurable structures, not hardcoded functions
-

What Is a Workflow? A Fundamental Redefinition

Beyond Traditional Workflow Systems

Traditional workflow systems like N8N or Apache Airflow operate on a simple principle: humans (or scheduled triggers) execute workflows that may contain AI components. Our system inverts this relationship entirely. In our architecture:

AI Agents Execute Workflows

Our intelligent agent doesn't live inside a workflow; instead, it discovers, initializes, and drives workflows to completion.

This inversion is philosophically and architecturally significant:

- **Agent as Orchestrator:** The agent maintains conversational context and decides which workflow to execute based on user intent
- **Workflow as Process Definition:** Each workflow defines a specific business process as a traversable graph
- **Separation of Concerns:** The agent handles natural language understanding and decision-making; workflows handle business logic and API integration

Workflow as Directed Cyclic Graph

A workflow in our system is fundamentally a Directed Cyclic Graph (DCG) structure with specific capabilities:

1. **Traversal Based on Actions:** The graph progresses based on user inputs, API responses, and conditional logic
2. **Agent Communication Interface:** Workflows communicate with the agent through structured messages
3. **Information Collection:** Workflows can collect required parameters from users through the agent
4. **Task Execution:** Workflows execute actual business operations through API calls
5. **State Management:** Workflows maintain execution state across multiple interaction turns

Why Directed Cyclic Graphs Matter

Unlike traditional acyclic flowcharts, our DCGs can loop back to previous nodes. This capability is crucial for real-world business processes:

- **Iterative Refinement:** Users can provide information incrementally, with the workflow cycling back to collect missing details
 - **Retry Logic:** Failed validations can return users to collection nodes with specific feedback
 - **Complex Conditionals:** Business processes often require revisiting previous steps based on new information
-

System Architecture: From JSON to Execution

The Data Flow: A Three-Tier System

User Request → Agent → Workflow Search → Workflow Initialization → Execution → Results

Step 1: Workflow Storage

- **MongoDB:** Stores complete workflow JSON definitions with all nodes, edges, and configurations
- **Milvus Vector DB:** Stores vector embeddings of workflow training text for semantic search

Step 2: Workflow Discovery

1. User makes a request to the agent
2. Agent calls the "Search for Available Operations" tool with a context-rich query
3. Tool converts query to vector using Azure OpenAI's text-embedding-3-large model
4. Semantic similarity search in Milvus returns top matching workflow IDs
5. Corresponding workflow names and descriptions fetched from MongoDB
6. Agent receives list of relevant workflows and selects appropriate one

Step 3: Workflow Initialization

1. Agent calls "Initialize Operation" tool with selected workflow name
2. Tool fetches complete workflow JSON from MongoDB
3. Workflow loaded into LangGraph session state as the "active workflow"
4. Workflow engine executes initial steps (typically parameter collection)
5. Tool returns workflow state and next required actions to agent

Step 4: Iterative Execution

1. Agent collects required information from user through natural conversation
 2. Agent calls "Submit Information" tool with user input
 3. Workflow processes input, validates if needed, stores in internal state
 4. Workflow progresses through graph based on conditions and logic
 5. Cycle continues until workflow reaches completion
-

The Workflow JSON Structure: Complete Reference

Core Metadata Section

Every workflow begins with essential metadata that defines its identity, purpose, and discovery characteristics.

`workflow_id`

- **Type:** String
 - **Required:** Yes
 - **Purpose:** Unique identifier for the workflow. This ID serves as the primary key linking MongoDB documents with Milvus vector records.
 - **Format:** Typically a UUID or other collision-resistant identifier
 - **Example:** "122cc5aa48224c06abc7505f0c9c651a"
-

`workflow_name`

- **Type:** String
 - **Required:** Yes
 - **Purpose:** Human-readable name that the agent uses to initialize the workflow. This name appears in search results and must be descriptive yet concise.
 - **Best Practices:** Use action-oriented names (e.g., "Create Incident Ticket" not "Ticket Creation Process")
 - **Example:** "Create Incident Ticket"
-

`workflow_description`

- **Type:** String
 - **Required:** Yes
 - **Purpose:** Natural language description of what the workflow accomplishes. The agent reads this description to determine if the workflow matches user intent.
 - **Characteristics:** Should be comprehensive yet concise, covering the objective, typical use cases, and any important constraints
 - **Example:** "Creates a new incident ticket in the ITSM system."
-

`training_text`

- **Type:** String
- **Required:** Yes

- **Purpose:** Text corpus used for vector embedding and semantic search. Contains example user queries that should trigger this workflow.
 - **Content Strategy:**
 - Include natural language variations of potential user requests
 - Incorporate synonyms and related phrases
 - Include the workflow description for contextual richness
 - Add common misspellings or alternative phrasings
 - **Example:** "I need to report a problem. Can you create a ticket for me? My laptop isn't working. There's an issue with my software. I want to report an incident. + Creates a new incident ticket in the ITSM system..."
-

executed_steps

- **Type:** Array of Strings
 - **Required:** Yes (initially empty)
 - **Default:** []
 - **Purpose:** Tracks which nodes have been executed during the current workflow instance. The workflow engine automatically manages this array as execution progresses.
 - **Engine Behavior:** When a node completes execution, its name is appended to this array. This enables replay prevention and state tracking.
-

current_step

- **Type:** String
 - **Required:** Yes
 - **Purpose:** Indicates which node is currently active. The workflow engine updates this value as the graph traverses.
 - **Initial Value:** Typically "`__SOFT_STORAGE__`" or the name of the node from where you want to start flow execution
 - **Engine Behavior:** After each node execution, the engine evaluates post-conditions and default steps to determine the new `current_step`
-

is_workflow_ended

- **Type:** Boolean
 - **Required:** Yes
 - **Default:** `false`
 - **Purpose:** Flag indicating whether the workflow has reached a terminal state. When `true`, no further execution occurs.
 - **Termination Triggers:**
 - Reaching a node with "`<--|end-of-flow|-->`" as its next step
 - Explicit termination via API response
 - Maximum retry count exceeded
-

steps

- **Type:** Object (key-value mapping)
- **Required:** Yes
- **Purpose:** Contains all nodes (steps) that comprise the workflow graph. Each key is a node name, and each value defines the node's behavior and configuration.
- **Structure:**

```
"steps": {  
  "node_name_1": { /* node definition */ },  
  "node_name_2": { /* node definition */ },  
  // ... additional nodes  
}
```

Node Types and Structures

Every node in a workflow belongs to one of two fundamental types, each serving distinct purposes in the execution graph.

Special System Nodes

__SOFT_STORAGE__

- **Type:** Parameter (specialized)
- **Purpose:** Global data sharing mechanism across workflows. Acts as a cross-workflow cache for frequently accessed information.
- **Use Cases:**
 - User authentication tokens
 - Frequently accessed user data (email, department, etc.)
 - Cross-workflow state sharing
 - Temporary data persistence between unrelated operations
- **Implementation Details:**
 - Always has an empty params dictionary initially
 - Workflows can read from and write to soft storage
 - Data persists for the duration of the user session
 - Enables "remember me" functionality across operations

<--|end-of-flow|-->

- **Type:** Terminal (special marker)
 - **Purpose:** Indicates workflow completion. When a node routes to this marker, the workflow ends successfully.
 - **Usage:** Set as `default_step` or `true_step` in post-conditions to terminate execution
-

Node Common Properties

All nodes, regardless of type, share these fundamental properties:

type

- **Type:** String Literal
 - **Required:** Yes
 - **Values:** "parameter" or "api_call"
 - **Purpose:** Determines the node's fundamental behavior and expected structure
-

default_step

- **Type:** String
 - **Required:** Yes
 - **Purpose:** Defines the next node to execute if no post-conditions are satisfied or if the node has no post-conditions.
 - **Special Values:** Can be another node name or <--|end-of-flow|-->
-

post_conditions

- **Type:** Array of Objects
 - **Required:** No
 - **Purpose:** Defines conditional branching logic based on node execution results. Enables dynamic graph traversal.
 - **Structure:** Array of condition objects, each containing:
 - Logical operator (for combining multiple conditions)
 - List of individual conditions to evaluate
 - Target node if conditions evaluate to true
-

Parameter Nodes: Information Collection Engine

Parameter nodes are responsible for collecting, validating, and storing user-provided information. They represent the primary interaction point between users and workflows.

Complete Parameter Node Structure

```
"parameter_node_name": {
  "type": "parameter",
  "params": {
    "parameter_key": {
      "value": "",
      "llm_key": "User-Friendly Name",
      "datatype": "str",
      "description": "What information this parameter represents",
      "validation": { /* validation rules */ },
      "available_options": ["Option1", "Option2"],
      "is_optional": false,
      "retry_count": 3
    }
  },
  "post_conditions": [ /* conditional branching */ ],
  "default_step": "next_node_name"
}
```

Parameter Definition Deep Dive

params Object

- **Structure:** Key-value mapping where each key is a parameter identifier and each value is a parameter definition object.

Parameter Definition Object Properties:

value

- **Type:** String (initially)
 - **Required:** Yes
 - **Default:** "" (empty string)
 - **Purpose:** Stores the user-provided value after successful validation. The workflow engine populates this field.
 - **Important:** Never pre-populate this field in the JSON definition.
-

llm_key

- **Type:** String
- **Required:** Yes
- **Purpose:** Human-readable name that appears to the LLM agent. This is how the agent identifies which parameter it's collecting.

- **Best Practices:** Use clear, descriptive names that match natural language expectations
 - **Example:** For a parameter storing email addresses, use "Email Address" not "user_email"
-

datatype

- **Type:** String
 - **Required:** Yes
 - **Purpose:** Defines the expected data type for validation and parsing. Uses Python typing notation as strings.
 - **Supported Types:**
 - Basic: "str", "int", "float", "bool"
 - Collections: "List[str]", "List[int]", "Dict[str, str]"
 - **Validation Behavior:** The workflow engine attempts to cast user input to this type before applying other validations.
-

description

- **Type:** String
 - **Required:** Yes
 - **Purpose:** Explains to the agent what information this parameter represents and any special considerations.
 - **Content Guidelines:** Include format expectations, business context, and usage examples
 - **Example:** "The user's corporate email address. Must end with @company.com. Used for authentication and notification."
-

validation

- **Type:** Object
- **Required:** No
- **Purpose:** Defines rules for validating user input before acceptance. Supports both local validation (regex, range checks) and remote validation (API calls).

Validation Object Structure:

```

"validation": {
  "function": "regex",
  "condition_key": "",
  "condition_source": "",
  "condition_step": "",
  "criteria": "^[a-zA-Z0-9._%+-]+@company\\.com$",
  "operator": "=",
  "start": "",
  "end": "",
  "api_call": {
    "api_endpoint": "validation/endpoint",
    "true_code": 200,
    "copy_params": [
      {
        "keys": [
          {
            "copy_from": "source_field",
            "copy_to": "target_field",
            "source": "response"
          }
        ],
        "step": "source_step"
      }
    ],
    "prefilled_params": {
      "routing_key": "validation_type"
    }
  }
}

```

Validation Components Explained:

- **function:** Built-in validation function name (e.g., "regex", "range", "len")
- **criteria:** Static value to compare against (e.g., regex pattern, list of valid options)
- **operator:** Comparison operator (e.g., "=", "in", ">")
- **Dynamic Value References:** `condition_key`, `condition_step`, `condition_source` allow comparing against values from other nodes
- **api_call:** Enables remote validation via API. The API must return structured responses with error codes

available_options

- **Type:** Array

- **Required:** No
 - **Purpose:** For categorical parameters, provides a list of valid choices. Can be static (hardcoded) or dynamic (populated from API responses).
 - **Dynamic Population:** Use `set_available_options` in API call nodes to populate this list at runtime
-

`is_optional`

- **Type:** Boolean
 - **Required:** Yes
 - **Default:** `false`
 - **Purpose:** Determines whether the parameter must be provided. Optional parameters can be skipped by users.
 - **Workflow Behavior:** Optional parameters that aren't provided receive null or empty values
-

`retry_count`

- **Type:** Integer
 - **Required:** No
 - **Purpose:** Limits how many times a user can attempt to provide valid input for this parameter.
 - **Default Behavior:** If unspecified, unlimited retries allowed
 - **Validation Failure Flow:** When retry count exceeded, workflow typically routes to an error handling node
-

API Call Nodes: Business Logic Execution

API call nodes represent the "action" components of workflows — they perform actual business operations by calling external systems.

Complete API Call Node Structure

```
"api_call_node_name": {
  "type": "api_call",
  "api_endpoint": "service/operation",
  "prefill_params": {
    "static_key": "static_value"
  },
  "copy_params": [ /* parameter mapping */ ],
  "response": {},
  "on_error": {
    "500": "error_handling_node",
    "701": "retry_node"
  },
  "is_silent_step": {
    "200": false,
    "708": true
  },
  "soft_storage_params": [ /* data persistence */ ],
  "retry_count": 3,
  "set_available_options": [ /* dynamic options */ ],
  "post_conditions": [ /* conditional routing */ ],
  "default_step": "next_node_name"
}
```

API Call Node Properties Deep Dive

api_endpoint

- **Type:** String
- **Required:** Yes
- **Purpose:** Defines the API endpoint to call (path only, not full URL). The base URL is configured at the system level.
- **Example:** "tickets/create" or "users/search"

prefill_params

- **Type:** Object
- **Required:** No
- **Purpose:** Static parameters included in every API request from this node. Useful for routing keys, operation types, or constant values.
- **Example:** {"operation_type": "CREATE_TICKET", "system": "ITSM"}

copy_params

- **Type:** Array of Objects
- **Required:** No
- **Purpose:** Maps values from workflow state into the API request payload. Enables dynamic parameter passing.

Copy Parameter Structure:

```
"copy_params": [
  {
    "keys": [
      {
        "copy_from": "source_parameter_name",
        "copy_to": "api_request_field_name",
        "source": "response"
      }
    ],
    "step": "source_node_name"
  }
]
```

Source Field Explained:

- Omitted or empty: Copy from parameter value directly
- "response": Copy from API response payload of the source node
- "response.field_name": Copy specific field from nested response

response

- **Type:** Object
- **Required:** Yes (initially empty)
- **Purpose:** Stores the complete API response after execution. The workflow engine populates this field.
- **Expected Structure:** All API responses must include:
 - `error_code`: Numeric status code (see Error Codes section)
 - `llm_feedback`: Natural language description of what happened

on_error

- **Type:** Object
- **Required:** No
- **Purpose:** Defines error-specific routing. Maps error codes to handling nodes.
- **Example:** `{"500": "system_error_node", "702": "auth_error_node"}`

is_silent_step

- **Type:** Object

- **Required:** No
 - **Purpose:** Controls whether the agent shares API execution results with users based on error codes.
 - **Logic:**
 - `true`: LLM feedback is NOT included in agent context (user doesn't see it)
 - `false`: LLM feedback IS included in agent context (user sees it)
 - **Use Case:** Background operations where success/failure doesn't affect user experience
-

soft_storage_params

- **Type:** Array of Objects
- **Required:** No
- **Purpose:** Persists data from this node's execution into the global soft storage for cross-workflow access.

Structure:

```
"soft_storage_params": [  
  {  
    "keys": [  
      {  
        "get_from": "response_field_name",  
        "set_to": "storage_key_name",  
        "source": "response"  
      }  
    ],  
    "step": "current_step"  
  }  
]
```

retry_count

- **Type:** Integer
 - **Required:** No
 - **Purpose:** Number of retry attempts for transient API failures. The workflow engine handles retry logic automatically.
 - **Retry Triggers:** Typically used for 5xx errors or network timeouts
-

set_available_options

- **Type:** Array of Objects
- **Required:** No
- **Purpose:** Dynamically populates `available_options` for parameter nodes based on API responses.

Structure:

```
"set_available_options": [  
  {  
    "set_from": "response.list_field",  
    "set_to": "target_parameter_name",  
    "set_step": "current_step"  
  }  
]
```

Post-Conditions: The Flow Control Language

Post-conditions represent the decision logic that determines how a workflow traverses its graph. They enable conditional branching based on execution results.

Complete Post-Condition Structure

```

"post_conditions": [
  {
    "logical_operator": "and",
    "conditions": [
      {
        "key": "status",
        "step": "previous_api_call",
        "source": "response",
        "function": "",
        "condition_key": "",
        "condition_source": "",
        "condition_step": "",
        "criteria": "success",
        "operator": "==",
        "start": "",
        "end": ""
      },
      {
        "key": "ticket_count",
        "step": "previous_api_call",
        "source": "response.count",
        "function": "len",
        "criteria": 5,
        "operator": ">"
      }
    ],
    "true_step": "handle_multiple_tickets"
  }
]

```

Post-Condition Components Explained

logical_operator

- **Type:** String Literal
- **Values:** "and", "or", "" (empty string)
- **Purpose:** Defines how multiple conditions are combined. Empty string indicates a single condition.
- **Current Limitation:** Only two conditions supported when using logical operators

conditions Array

Contains one or more condition objects to evaluate.

Condition Object Properties:

key

- **Type:** String
 - **Required:** Yes
 - **Purpose:** Identifies which value to evaluate. Corresponds to a parameter name or response field.
-

step

- **Type:** String
 - **Required:** Yes
 - **Purpose:** Specifies which node contains the value to evaluate.
-

source

- **Type:** String
 - **Required:** No
 - **Purpose:** Indicates where within the step to find the value.
 - **Values:**
 - Omitted: Value from parameter itself
 - "response": Entire API response object
 - "response.field_name": Specific field in API response
-

function

- **Type:** String Literal
 - **Required:** No
 - **Purpose:** Applies a transformation or check before comparison.
 - **Supported Functions:**
 - "range": Check if value is within numeric range
 - "len": Get length of string or list
 - "regex": Apply regular expression pattern
 - "is_empty" / "is_not_empty": Check for empty values
 - "is_none" / "is_not_none": Check for null values
 - "is_subset": Check if all elements are in another list
 - "is_date_time_format_valid": Validate date/time strings
-

condition_key, condition_step, condition_source

- **Type:** String
- **Required:** No
- **Purpose:** Alternative to `criteria` for dynamic comparison values. These three fields identify a value from another node to compare against.

`criteria`

- **Type:** Various (depends on operator)
- **Required:** Conditional (required if not using dynamic condition values)
- **Purpose:** Static value to compare against.

`operator`

- **Type:** String Literal
- **Required:** Yes
- **Purpose:** Defines the comparison operation.
- **Supported Operators:** "=", "!=", ">", "<", ">=", "<=", "in", "not in"

`start and end`

- **Type:** String or Number
- **Required:** Only for "range" function
- **Purpose:** Define the inclusive range boundaries for numeric comparisons.

`true_step`

- **Type:** String
- **Required:** Yes
- **Purpose:** The node to execute if all conditions evaluate to true.
- **Special Values:** Can be another node name or `<--|end-of-flow|-->`

Error Handling Framework

Standardized Error Codes

All API responses must include an `error_code` field with one of these standardized values:

Code	Meaning	Typical Response
200	Success	Operation completed successfully

Code	Meaning	Typical Response
500	Server Error	Backend system unavailable or internal error
701	Validation Error	Input data failed validation rules
702	Authentication Error	Invalid or expired credentials
704	Not Found/Invalid	Requested resource doesn't exist
706	Permission Denied	User lacks required permissions
708	Business Logic Error	Operation violates business rules

Error Propagation Model

1. **API Level:** Backend systems return structured errors with codes
 2. **Workflow Level:** API call nodes receive and process error responses
 3. **Routing Level:** `on_error` mappings determine error-specific handling
 4. **User Level:** `llm_feedback` provides user-appropriate error messages
-

Workflow Engine: Execution Mechanics

The Execution Loop

The workflow engine operates on a simple but powerful state machine:

1. Load workflow JSON into state
2. Identify `current_step`
3. Execute step based on type:
 - Parameter: Collect from user via agent
 - API Call: Execute HTTP request
4. Process results:
 - Update workflow state
 - Evaluate post-conditions
 - Determine next step
5. Repeat until `is_workflow_ended = true`

State Management

Session State Includes:

- Complete workflow JSON structure
- Current execution position (`current_step`)
- Execution history (`executed_steps`)
- Collected parameter values
- API response payloads
- Soft storage cache

State Persistence: Managed by LangGraph session management, enabling:

- Resume interrupted workflows
- Long-running process support
- Multi-turn conversation context

Agent-Workflow Communication Protocol

Workflows and agents communicate through a structured message protocol:

Workflow → Agent Messages:

- "need_parameter": Request specific information from user
- "execution_result": Report API call outcomes
- "validation_failed": Indicate invalid input with reasons
- "workflow_completed": Signal successful completion

Agent → Workflow Messages:

- "parameter_value": Provide user input for requested parameters
- "initiate_workflow": Start new workflow execution
- "cancel_workflow": Terminate ongoing workflow

Advanced Patterns and Use Cases

Multi-Step Validation Chains

```
"validate_user_access": {
  "type": "api_call",
  "api_endpoint": "auth/validate",
  "copy_params": [
    {
      "keys": [
        {
          "copy_from": "user_email",
          "copy_to": "email",
          "step": "collect_user_email"
        }
      ]
    }
  ],
  "on_error": {
    "702": "request_new_credentials",
    "706": "escalate_to_admin"
  },
  "post_conditions": [
    {
      "conditions": [
        {
          "key": "error_code",
          "step": "validate_user_access",
          "source": "response",
          "criteria": 200,
          "operator": "=="
        }
      ],
      "true_step": "proceed_with_operation"
    }
  ],
  "default_step": "access_denied"
}
```

Dynamic Option Population

```
"fetch_departments": {
  "type": "api_call",
  "api_endpoint": "hr/departments",
  "set_available_options": [
    {
      "set_from": "response.departments",
      "set_to": "user_department",
      "set_step": "fetch_departments"
    }
  ],
  "default_step": "collect_user_info"
}

"collect_user_info": {
  "type": "parameter",
  "params": {
    "user_department": {
      "value": "",
      "llm_key": "Department",
      "datatype": "str",
      "description": "User's department for ticket routing",
      "available_options": [], /* Dynamically populated */
      "is_optional": false
    }
  },
  "default_step": "create_ticket"
}
```

Cross-Workflow Data Sharing

```
"authenticate_user": {
  "type": "api_call",
  "api_endpoint": "auth/login",
  "copy_params": [
    {
      "keys": [
        {
          "copy_from": "email",
          "copy_to": "username",
          "step": "collect_email"
        },
        {
          "copy_from": "password",
          "copy_to": "password",
          "step": "collect_password"
        }
      ]
    }
  ],
  "soft_storage_params": [
    {
      "keys": [
        {
          "get_from": "response.auth_token",
          "set_to": "user_auth_token",
          "source": "response"
        },
        {
          "get_from": "response.user_id",
          "set_to": "current_user_id",
          "source": "response"
        }
      ]
    },
    {
      "step": "authenticate_user"
    }
  ],
  "default_step": "main_menu"
}

"access_protected_resource": {
  "type": "api_call",
  "api_endpoint": "data/protected",
  "prefill_params": {
```

```
    "auth_token": "" /* Will be filled from soft storage */
  },
  "copy_params": [
    {
      "keys": [
        {
          "copy_from": "user_auth_token",
          "copy_to": "auth_token",
          "step": "__SOFT_STORAGE__"
        }
      ]
    }
  ],
  "default_step": "display_data"
}
```

Current Limitations and Future Directions

Architectural Constraints

1. **Single Validation Constraint:** Parameters can only have one validation function applied. Cannot chain multiple validations (e.g., "must be email AND must be verified").
2. **Condition Complexity Limit:** Post-conditions with logical operators support only two conditions. No nested conditions or complex boolean logic.
3. **Workflow Isolation:** Workflows cannot call or include other workflows. Leads to code duplication for common patterns.
4. **Data Injection Limitations:** Only user input via parameter collection can populate workflow data. No support for scheduled injection or external triggers.

Scalability Considerations

1. **Vector Search Quality:** Workflow discovery depends entirely on training text quality and embedding accuracy.
2. **State Management:** No automatic context summarization or compression for long conversations.
3. **Error Recovery:** Limited built-in recovery mechanisms for failed API calls beyond simple retries.

Future Enhancement Areas

1. **Workflow Composition:** Enable workflows to call other workflows as subroutines.
2. **Advanced Validation Chains:** Support multiple validation rules with custom error messages.
3. **Event-Driven Triggers:** Allow workflows to be initiated by system events, not just user requests.
4. **Scheduled Execution:** Support time-based or condition-based workflow scheduling.
5. **Real-time Data Streams:** Integrate WebSocket or streaming data sources into workflows.

Best Practices and Design Guidelines

Workflow Design Principles

1. **Single Responsibility:** Each workflow should accomplish one primary business objective.
2. **Progressive Disclosure:** Collect only necessary information at each step.
3. **Defensive Validation:** Validate early, validate often, provide clear error messages.
4. **Graceful Degradation:** Handle all expected error cases with user-friendly responses.
5. **State Awareness:** Use soft storage judiciously for cross-workflow data sharing.

JSON Structure Guidelines

1. **Descriptive Naming:** Use clear, consistent naming conventions for nodes and parameters.
2. **Modular Design:** Group related parameters in single collection nodes when appropriate.
3. **Documentation:** Include comprehensive descriptions for all workflows and parameters.
4. **Error Handling:** Always define `on_error` mappings for API call nodes.
5. **Testing Considerations:** Design workflows with testability in mind — avoid overly complex conditional chains.

Performance Optimization

1. **Minimize API Calls:** Cache frequently accessed data in soft storage.
2. **Batch Operations:** Combine related API calls when possible.
3. **Early Exit:** Use post-conditions to exit workflows quickly when conditions aren't met.
4. **Parameter Ordering:** Collect mandatory parameters before optional ones.
5. **Validation Sequencing:** Perform cheap validations (format checks) before expensive ones (API calls).

Conclusion: The Workflow-Centric Architecture

The workflow engine represents a fundamental architectural innovation in agentic systems. By externalizing business logic into configurable, discoverable, executable graphs, we achieve the seemingly contradictory goals of product consistency and client customization.

This documentation serves not just as a technical reference but as a philosophical guide. The workflow-centric approach embodies several core principles:

1. **Separation of Intelligence and Execution:** The agent handles "what," workflows handle "how."
2. **Configuration Over Code:** Business logic lives in structured data, not compiled binaries.
3. **Discoverability Through Embedding:** Semantic search enables scale without context window inflation.
4. **Stateful Conversations:** Workflows maintain context across multi-turn interactions.

5. **Graceful Evolution:** New capabilities added as new workflows, not system rewrites.

As the system grows beyond its current seventy workflows, these principles will guide its evolution. Future engineers inheriting this system should understand that its apparent simplicity — the clean separation between agent and workflow — is the result of hard-won insights from multiple failed architectures. When tempted to add complexity, consider first whether the need can be met by a new workflow configuration rather than a new system component.

The workflow engine is more than a technical component; it's the embodiment of our approach to scalable, adaptable, maintainable agentic automation. May this documentation serve as both map and compass for those who continue its development.